ACI COMPANION READER · AIMLCZG557

Ant Colony Optimization & Neural Architecture Search

Session 6 — From swarm intelligence to self-designing AI

Session 6 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a plain-English explanation. Every slide example is worked out step by step. Five box types guide you through each concept. The **blue** box states the core idea. The **amber** box ties it to real life before any math. The **green** box solves a problem with real numbers. The **red** box flags the common mistake. The **purple** box is the one line to remember.

1 What this session covers

This session has two big parts.

1. **Ant Colony Optimization (ACO).** We look at how simulated ants, leaving digital scent trails, can solve hard routing problems like the Travelling Salesman Problem.
2. **Neural Architecture Search (NAS) and CoDeepNEAT.** We ask: can a computer design its own neural networks? The answer is yes — using evolution. We trace the lineage from plain NEAT, through DeepNEAT, to CoDeepNEAT, and run through a full worked example.

2 Ant Colony Optimization: the big idea

How do you find the shortest path through a city when there are too many routes to check one by one? Real ants solved this problem long before computers existed.

☞ The intuition

ACO (Ant Colony Optimization) is a search algorithm inspired by how real ants find food. Each artificial ant builds a candidate solution. It leaves a chemical signal, called a **pheromone**, on the path it took. Good paths collect more pheromone. Over many rounds, the colony converges on the best route.

★ Everyday picture

Imagine a food court with many corridors. Hundreds of people walk from the entrance to the counter. Some try the long route; some try the shortcut. People notice the crowds ahead and tend to follow the busier path. The short, fast corridor gets more foot traffic. Over time almost everyone uses it. That is pheromone in human form.

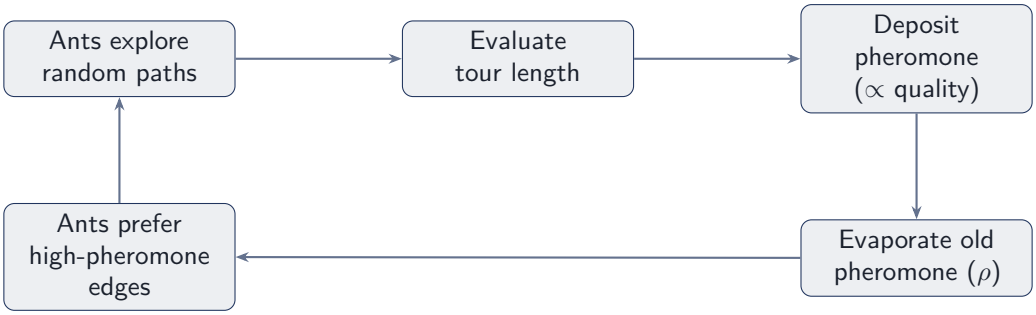


Figure 1: *
The ACO feedback loop: good routes earn more pheromone, attracting future ants, while evaporation prevents early lock-in.

2.1 ACO parameters

Every ACO run uses a handful of numbers. Knowing what each one does lets you tune the algorithm.

Symbol	Name	What it controls
N	Number of ants	Total ants per iteration; $N > 1$
τ_0	Initial pheromone	Starting scent on every edge; set to 0
τ_{ij}	Pheromone on (i, j)	Accumulated scent on the edge from city i to j
η_{ij}	Heuristic value	$1/d_{ij}$; short edges get high η
α	Pheromone weight	How strongly ants follow scent ($\alpha > 0$)
β	Distance weight	How strongly ants prefer short edges ($\beta > 0$)
ρ	Evaporation rate	How fast scent fades; $0 < \rho < 1$
visit_k	Visit list of ant k	Cities ant k has already visited
Q	Pheromone constant	Scales the reward deposited on a completed tour
f_k	Tour cost of ant k	Total distance ant k travelled

2.2 ACO steps

The algorithm runs three phases per iteration.

Initialization. Place all N ants at the starting city. Set α, β, ρ. Set $\tau_0 = 0$ on every edge.

Construction. Each ant picks its next city using the **Next Transition Probability (NTP)**. For ant k currently at city i, the probability of moving to city j is:

$$NTP_{ij} = \frac{(\tau_{ij})^{\alpha}\,(\eta_{ij})^{\beta}}{\sum_{h \notin \text{visit}_k} (\tau_{ih})^{\alpha}\,(\eta_{ih})^{\beta}} \tag{1}$$

Read it in two pieces. The numerator scores city j: pheromone on that edge to the power α, times the heuristic (inverse distance) to the power β. The denominator sums the same score over every city not yet visited. Dividing gives a probability that sums to 1 over all unvisited cities.

Pheromone update. After all ants finish their tours, each edge is updated:

$$\tau_{ij}^{\text{new}} = (1 - \rho)\,\tau_{ij}^{\text{old}} + \Delta\tau_{ij}^{k} \tag{2}$$

The term $(1-\rho)\,\tau_{ij}^{\text{old}}$ decays the old scent. The term $\Delta\tau_{ij}^{k}$ is the fresh deposit by ant k:

$$\Delta\tau_{ij}^{k} = \begin{cases} Q/f_k & \text{if ant } k \text{ used edge } (i,j) \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

Ants that completed short tours (small f_k) deposit more pheromone per edge because Q/f_k is large. That is the positive feedback: good routes attract more future ants.

✗ Watch out

ACO stops after a fixed number of iterations. If that limit is reached before an ant completes its tour, the ant is "dropped" and its partial path is discarded. Set the iteration limit generously, or add a completion check before stopping.

Where this shows up in ML. ACO is used to tune neural network hyperparameters, select features, and route data in communication networks. This update works like a reward signal in reinforcement learning. Edges used by successful agents grow stronger. This mirrors how actions that led to high reward get reinforced.

✓ Key takeaway

ACO replaces exhaustive search with collective learning. Short, cheap paths collect more pheromone and attract more ants. Evaporation stops the algorithm from locking onto a suboptimal route early.

3 Worked example: Travelling Salesman with ACO

The **Travelling Salesman Problem** (TSP) is a classic routing puzzle. You have n cities with known distances between every pair. Find the shortest tour that visits each city exactly once and returns to the start.

Setup. Cities: 1, 2, 3, 4, 5. Ants $N = 5$ (one per city). Start city: 4. Parameters: $\alpha = 0.5$, $\beta = 0.75$, $\rho = 0.1$, $Q = 100$.

The distance matrix d_{ij} (symmetric):

	1	2	3	4	5
1	0	65	52	69	72
2	65	0	39	28	43
3	52	39	0	72	62
4	69	28	72	0	49
5	72	43	62	49	0

Initial pheromone values (given in the slides):

$\tau_{12} = \tau_{21} = 0.54$	$\tau_{23} = \tau_{32} = 0.53$	$\tau_{35} = \tau_{53} = 0.90$
$\tau_{13} = \tau_{31} = 0.53$	$\tau_{24} = \tau_{42} = 0.39$	$\tau_{45} = \tau_{54} = 0.68$
$\tau_{14} = \tau_{41} = 0.35$	$\tau_{25} = \tau_{52} = 0.18$	
$\tau_{15} = \tau_{51} = 0.24$	$\tau_{34} = \tau_{43} = 0.32$	

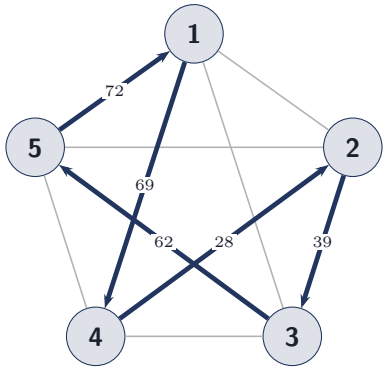


Figure 2: *
The 5-city TSP network. Bold arrows show the tour the ant finds: $4 \to 2 \to 3 \to 5 \to 1 \to 4$, total distance 270.

✏ Worked example: TSP step 1 — move from city 4

The ant starts at city 4. Unvisited cities: 1, 2, 3, 5. We compute NTP_{4j} for each. $\eta_{4j} = 1/d_{4j}$, raised to power $\beta = 0.75$. Each numerator is $(\tau_{4j})^{0.5} \cdot (1/d_{4j})^{0.75}$.

Step 1. Compute each numerator.

$$\begin{aligned}
\text{To city 1:} \quad & (0.35)^{0.5} \cdot (1/69)^{0.75} = 0.0247 \\
\text{To city 2:} \quad & (0.39)^{0.5} \cdot (1/28)^{0.75} = 0.0513 \\
\text{To city 3:} \quad & (0.32)^{0.5} \cdot (1/72)^{0.75} = 0.0229 \\
\text{To city 5:} \quad & (0.68)^{0.5} \cdot (1/49)^{0.75} = 0.0445
\end{aligned}$$

Step 2. Sum the numerators (denominator).

$$D = 0.0247 + 0.0513 + 0.0229 + 0.0445 = 0.1434$$

Step 3. Compute each probability.

$$\begin{aligned}
P_{41} &= 0.0247/0.1434 = 0.1723 \\
P_{42} &= 0.0513/0.1434 = \mathbf{0.3577} \\
P_{43} &= 0.0229/0.1434 = 0.1596 \\
P_{45} &= 0.0445/0.1434 = 0.3104
\end{aligned}$$

Step 4. Pick the next city. P_{42} is largest. The ant moves from city 4 to city **2**. $\text{visit}_k = [4, 2]$.

Step 5. Update pheromone on edge (4,2). Deposit: $\Delta\tau_{42} = Q/d_{42} = 100/28 = 3.571$.

$$\tau_{42}^{\text{new}} = (1 - 0.1) \times 0.39 + 3.571 = 0.351 + 3.571 = \mathbf{3.922}$$

All other edges simply evaporate by factor 0.9. For example: $\tau_{12}^{\text{new}} = 0.9 \times 0.54 = 0.486$.

Step 6. Sense-check. Edge $(4, 2)$ is the shortest from city 4 ($d = 28$). It should have the highest probability and it does.

✏ Worked example: TSP step 2 — move from city 2 (visit: 4-2)

Unvisited: cities 1, 3, 5. Updated pheromones after step 1: $\tau_{12} = 0.486$, $\tau_{23} = 0.477$, $\tau_{25} = 0.162$.

Step 1. Compute numerators from city 2.

$$\begin{aligned} \text{To 1:} \quad & (0.486)^{0.5} \cdot (1/65)^{0.75} = 0.0305 \\ \text{To 3:} \quad & (0.477)^{0.5} \cdot (1/39)^{0.75} = 0.0443 \\ \text{To 5:} \quad & (0.162)^{0.5} \cdot (1/43)^{0.75} = 0.0240 \end{aligned}$$

Step 2. Denominator. $D = 0.0305 + 0.0443 + 0.0240 = 0.0988$.

Step 3. Probabilities.

$$P_{21} = 0.3086, \quad P_{23} = \mathbf{0.4485}, \quad P_{25} = 0.2429$$

Step 4. Ant moves from 2 to city 3. $\text{visit}_k = [4, 2, 3]$. Update τ_{23}:

$$\tau_{23}^{\text{new}} = 0.9 \times 0.477 + 100/39 = 0.4293 + 2.564 = \mathbf{2.993}$$

✏ Worked example: TSP steps 3–5 — completing the tour

Step 1. Step 3: from city 3, visit = [4-2-3]. Unvisited: 1 and 5.

$$\begin{aligned} P_{31} &= \frac{(0.4293)^{0.5}(1/52)^{0.75}}{(0.4293)^{0.5}(1/52)^{0.75} + (0.729)^{0.5}(1/62)^{0.75}} \\ &= \frac{0.0338}{0.0725} = 0.4668 \end{aligned}$$

$$P_{35} = \frac{0.0386}{0.0725} = \mathbf{0.5332}$$

Ant moves to city **5**. Update τ_{35}: $0.9 \times 0.729 + 100/62 = 0.656 + 1.613 = \mathbf{2.269}$.

Step 2. Step 4: from city 5, visit = [4-2-3-5]. Only city 1 remains. The ant must move to city **1**. Update τ_{51}: $0.9 \times 0.175 + 100/72 = 0.158 + 1.389 = \mathbf{1.546}$.

Step 3. Step 5: from city 1, return to origin city 4. The tour is complete: $4 \to 2 \to 3 \to 5 \to 1 \to 4$. Tour cost:

$$f_k = d_{42} + d_{23} + d_{35} + d_{51} + d_{14} = 28 + 39 + 62 + 72 + 69 = \mathbf{270}$$

Update τ_{14}: $0.9 \times 0.230 + 100/69 = 0.207 + 1.449 = \mathbf{1.656}$.

Step 4. Sense-check. The tour $4 \to 2 \to 3 \to 5 \to 1 \to 4$ uses the shortest edges from city 4 first (28, 39, 62). That is plausible for a greedy, pheromone-guided ant. Over many iterations all five ants refine the pheromone map further.

4 Neural Architecture Search: why it matters

Designing a deep neural network by hand is hard. The space of possible architectures is enormous: how many layers, which types, which connections, what hyperparameters. Human experts use trial and error, which is slow and misses many good designs.

☞ The intuition

Neural Architecture Search (NAS) lets a computer search the space of possible network designs automatically. Instead of a human choosing "3 conv layers, then a dense layer", an algorithm explores many candidates and picks the best one.

★ Everyday picture

Think of NAS like designing a Lego model with a robot's help. The robot tries millions of combinations and tests each one. It keeps whatever holds up best. No human expert needed.

Where this shows up in ML. NAS produced EfficientNet, NASNet, and many state-of-the-art architectures. It reduced the human bottleneck in applied deep learning. AutoML pipelines use NAS to deploy models without hand-tuning.

✓ Key takeaway

NAS moves architecture design from art to computation. The search algorithm replaces human intuition about layer choices.

5 Neuroevolution and NEAT

Neuroevolution means using genetic algorithms to evolve neural networks. The idea: treat a network's structure as a chromosome. Breed the best networks, mutate the offspring, repeat.

The core loop:

1. Create a population of networks.
2. Train each network on the task.
3. Measure performance — this score is called **fitness**.
4. Select the best networks.
5. Generate new networks by crossover and mutation.
6. Go back to step 2.

NEAT (NeuroEvolution of Augmenting Topologies) is a specific neuroevolution method. In NEAT, a network is a graph: nodes are neurons, edges are connections. NEAT starts with minimal networks and grows them over time. Each new structural change gets a unique **innovation number**. These numbers align matching genes during crossover so children are valid networks. Similar networks group into **species** and compete within their group, not against very different networks. This preserves diversity.

✗ Watch out

NEAT works well for small networks. It cannot build deep, layered architectures like CNNs or LSTMs. Modern image classifiers and language models need layer-wise design. NEAT alone is not enough for them.

Where this shows up in ML. NEAT was used to evolve controllers for game-playing agents.

It also appears in robotics and control tasks with small networks.

✓ Key takeaway

NEAT evolves network topology automatically. Innovation numbers ensure crossover produces valid offspring. But NEAT is limited to shallow, neuron-level structures.

6 DeepNEAT: scaling up to layers

DeepNEAT extends NEAT to deep networks. The key change: in NEAT each node is a single neuron; in DeepNEAT each node represents a whole layer. A node stores the layer type (Convolution, Dense, LSTM) and its hyperparameters (number of filters, kernel size, activation function). Edges show how layers connect. Unlike NEAT, edges in DeepNEAT do *not* store weights — weights are learned by gradient descent.

The assembly process:

1. Traverse the chromosome graph step by step.
2. For each node, create the corresponding neural network layer.
3. Connect all layers as the graph specifies.
4. When multiple inputs arrive at one node, combine via concatenation or summation.
5. If sizes do not match, apply pooling or downsampling.

✗ Watch out

DeepNEAT generates random-looking, unstructured networks. Modern high-performance models like ResNet use repeated, modular blocks. DeepNEAT does not reuse learned sub-structures. That gap led to CoDeepNEAT.

Where this shows up in ML. DeepNEAT showed that evolutionary methods could produce competitive deep networks on image tasks. It motivated the modular NAS approaches used in AutoML today.

7 CoDeepNEAT: co-evolving blueprints and modules

CoDeepNEAT (Co-evolving Deep NeuroEvolution of Augmenting Topologies) fixes the modularity problem by separating evolution into two populations that evolve together.

☞ The intuition

CoDeepNEAT splits the work into two parts. A **blueprint** is the overall wiring diagram — it says which types of components go where, not what they are. A **module** is a small, reusable neural sub-network. The final network is assembled by slotting specific modules into the blueprint's slots. Both blueprints and modules evolve, each improving the other.

★ Everyday picture

Think of building a car. The blueprint says: engine here, gearbox there, wheels at the corners. The modules are the actual engine, gearbox, and wheels, built and improved independently. You can swap a better engine into the same blueprint without touching the rest. CoDeepNEAT does exactly that with neural network components.

7.1 The genotype: blueprints and modules

Blueprint (macro-architecture). A blueprint is a graph. Each node holds a module *species ID* — a placeholder, not a specific layer. Edges define data flow between modules. The blueprint captures the high-level arrangement.

Module (micro-architecture). A module is a small neural network. It defines layer types (Conv, Dense, LSTM) and their hyperparameters (filters, neurons, activation function). Modules evolve in subpopulations. The same module can appear at multiple positions in a blueprint: that is reuse.

7.2 Phenotype construction

The **phenotype** is the final, trainable network. It is built in five steps:

1. Select a blueprint from the blueprint population.
2. For each node, pick a module from the corresponding module species.
3. Replace each blueprint node with its chosen module.
4. Connect modules following the blueprint's edge topology.
5. The result is a complete deep network ready for training.

Innovation numbers. Each structural component gets a unique innovation number. During crossover, matching components align by number. Children are valid architectures.

7.3 The evolutionary mechanics

Fitness Train the assembled network for 5–10 epochs. Measure validation accuracy. If the network classifies 90 out of 100 images correctly, fitness = 90%.

Selection Higher fitness means a higher chance of selection. Selection probability for network i: $P_i = f_i / \sum_j f_j$.

Crossover Two parents combine. Matching innovation numbers align corresponding layers. Extra layers from one parent are inherited if they help.

Mutation Random changes: add a layer, add a skip connection, change a hyperparameter (e.g., filter size $3 \times 3 \to 5 \times 5$).

Speciation Architectures group by structural similarity. Competition within species preserves diversity.

Shared fitness. When an assembled network scores 92%, that score goes to both the blueprint and every module that took part. Both get credit. This drives simultaneous improvement in module quality and blueprint design across generations.

Where this shows up in ML. CoDeepNEAT and its descendants are used in AutoML pipelines.

Google's NASNet and EfficientNet use similar bilevel search ideas. The bilevel split also mirrors LoRA: a frozen pre-trained structure (blueprint) plus small learned adapters (modules).

✓ Key takeaway

CoDeepNEAT separates *what components exist* (modules) from *how they are arranged* (blueprints). Both populations co-evolve. Shared fitness ties their fates together. The result is modular, reusable, and deep.

8 Worked example: CoDeepNEAT on image captioning

A company wants to auto-generate captions for images. We run one complete CoDeepNEAT iteration.

✏ Worked example: Step 1 — Initialization

Three modules and two blueprints.

Module population:

Module ID	Structure
M1	CNN Feature Extractor
M2	LSTM Layer
M3	Dense + Softmax

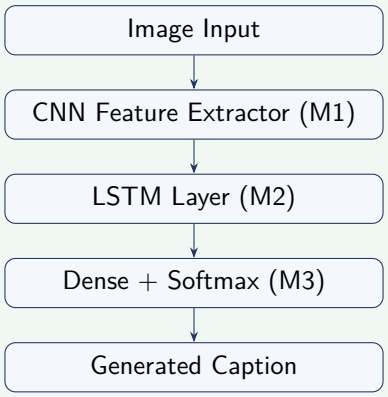
Blueprint population:

$$B_1 = [M1 \to M2 \to M3], \qquad B_2 = [M1 \to M1 \to M3]$$

B_1 pipes images through a CNN, then an LSTM, then a classifier. B_2 uses two CNN stages before the classifier.

✏ Worked example: Step 2 — Phenotype construction

Replace each blueprint node with a module. Using blueprint B_1, the assembled network flows:



This structure was not manually designed. Evolution assembled it.

✏ Worked example: Step 3 — Fitness evaluation

Train each assembled network for a few epochs. Score each with the **BLEU score**. BLEU runs from 0 to 1 and measures how well a caption matches a reference. Higher is better.

Network	BLEU Score	Fitness
N1	0.71	0.71
N2	0.88	0.88
N3	0.64	0.64

Best network this generation: $N2 = 0.88$.

✏ Worked example: Step 4 — Fitness-proportionate selection

We use roulette-wheel selection. Each network's chance equals its fitness divided by total fitness.

Step 1. Total fitness.

$$T = 0.71 + 0.88 + 0.64 = 2.23$$

Step 2. Selection probabilities.

$$P(N1) = 0.71/2.23 = \mathbf{0.318}$$
$$P(N2) = 0.88/2.23 = \mathbf{0.394}$$
$$P(N3) = 0.64/2.23 = \mathbf{0.287}$$

Step 3. Select parents. $N2$ has the highest probability. $N1$ has the second highest. They become parents for crossover.

Step 4. Sense-check. Sum: $0.318 + 0.394 + 0.287 = 0.999 \approx 1.0$. Good. Better networks get a bigger slice of the wheel.

✏ Worked example: Step 5 — Crossover

Parent architectures:

Parent 1 (N2): CNN → LSTM → Dense

Parent 2 (N1): CNN → CNN → Dense

Apply one-point crossover after the first module.

Step 1. Split each parent after module 1. P1 prefix: [CNN]; P1 suffix: [LSTM → Dense]. P2 prefix: [CNN]; P2 suffix: [CNN → Dense].

Step 2. Swap suffixes.

Child 1: CNN → CNN → Dense

Child 2: CNN → LSTM → Dense

Step 3. Interpret. Child 1 has two CNN stages (deeper feature extraction). Child 2 uses CNN then LSTM (feature extraction then sequence modelling).

✏ **Worked example: Step 6 — Mutation**

Step 1. Mutate Child 1. Change the second CNN's filter size from 3×3 to 5×5:

$$\text{Child 1 after:} \quad \text{CNN}(3 \times 3) \rightarrow \text{CNN}(5 \times 5) \rightarrow \text{Dense}$$

Larger filters capture broader patterns such as full face regions.

Step 2. Mutate Child 2. Increase the LSTM hidden size to 256 neurons:

$$\text{Child 2 after:} \quad \text{CNN} \rightarrow \text{LSTM}(256) \rightarrow \text{Dense}$$

More LSTM neurons improve sequence memory for caption generation.

Step 3. Next generation population.

Network	Architecture
Parent N1	CNN → LSTM → Dense
Parent N2	CNN → CNN → Dense
Child 1	CNN(3 × 3) → CNN(5 × 5) → Dense
Child 2	CNN → LSTM(256) → Dense

✏ **Worked example: Step 7 — Next-generation fitness evaluation**

Train and evaluate all four networks again.

Network	Accuracy
Parent N1	88%
Parent N2	84%
Mutated Child 1	91%
Mutated Child 2	**94%**

Mutated Child 2 (CNN → LSTM(256) → Dense) is now the best architecture. The LSTM size increase improved sequence learning. Evolution found this without any human decision.

9 The full CoDeepNEAT pipeline: cat vs. dog classification

The slides also trace what each layer does inside an evolved network for image classification.

9.1 Input as numbers

A 32×32 colour image has $32 \times 32 \times 3 = 3072$ values. Each pixel has three channels (R, G, B), each from 0 to 255. Normalize to $[0, 1]$ by dividing by 255. Example: pixel $(120, 80, 60) \rightarrow (0.47, 0.31, 0.23)$. At this point the system sees only numbers, not a cat.

9.2 The evolved architecture

CoDeepNEAT evolved these modules (not chosen by any human):

- Module 1: Conv(3×3) + ReLU
- Module 2: Conv(5×5) + Pool
- Module 3: Dense layer

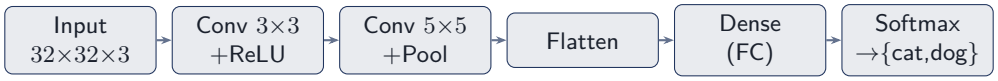


Figure 3: *
The CoDeepNEAT-evolved pipeline for cat vs. dog: two convolution modules (evolved) feed a dense classifier ending in a two-class softmax.

9.3 What each layer does

Conv 3×3 Applies a 3×3 filter. Detects edges: ears, body outlines. Outputs a feature map highlighting edges.

ReLU $f(x) = \max(0, x)$. Removes negative values. Keeps only strong, positive edge responses.

Conv 5×5 Combines edges into textures: fur patterns, eye regions. Now the network distinguishes cat-like from dog-like texture.

Max Pool 2×2
Reduces spatial size. Keeps the strongest value in each 2×2 block. Answers "Is the feature present?" not "Where exactly?"

Dense Combines textures into parts: triangular ears, small nose, whisker lines.

Flatten + Dense
Flattens all features to a vector. Learns rules like "whiskers + triangular ears = likely cat."

9.4 Softmax output

The softmax function turns raw scores (logits) into probabilities. For class y_i with raw score z_i:

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{4}$$

Suppose cat logit $= 2.5$ and dog logit $= 0.3$:

$$e^{2.5} = 12.18, \quad e^{0.3} = 1.35 \tag{5}$$

$$P(\text{cat}) = \frac{12.18}{12.18 + 1.35} = \frac{12.18}{13.53} \approx 0.90 \tag{6}$$

$$P(\text{dog}) = \frac{1.35}{13.53} \approx 0.10 \tag{7}$$

Final prediction: **Cat (90% confidence)**. (The slides use slightly different logits, giving 92%.)

9.5 Backpropagation when wrong

If the true label is Dog but the prediction is Cat, the error is:

$$\text{Loss} = \text{cross-entropy}(-\log P(\text{dog}))$$

Gradient descent updates all weights to reduce this loss. Over many training images the weights converge so the network classifies correctly.

Where this shows up in ML. This layer hierarchy is the foundation of every CNN: ResNet, VGG, EfficientNet. CoDeepNEAT automates the choice of which layers to use and how to connect them, removing the need for manual architecture design.

✗ Watch out

The network does not see animals the way humans do. It processes edges, then textures, then parts, then a label. A cleverly crafted adversarial input can fool it by tweaking a few pixels while still looking normal to a human eye.

10 Comparison: NEAT, DeepNEAT, and CoDeepNEAT

Property	NEAT	DeepNEAT	CoDeepNEAT
Node represents	single neuron	one full layer	module species (placeholder)
Evolves weights	yes	no (gradient descent)	no (gradient descent)
Handles deep CNNs	no	yes (unstructured)	yes (structured, modular)
Reuses components	no	no	yes (modules reused across blueprint)
Two co-evolving populations	no	no	yes (blueprints + modules)
Main use	small control tasks	medium-depth nets	large structured deep networks

Glossary & symbols

Key terms.

ACO Ant Colony Optimization; simulated ants deposit pheromone to guide search toward good solutions.

TSP Travelling Salesman Problem; find the shortest tour visiting every city exactly once.

pheromone Digital scent on a graph edge; accumulates on good paths and evaporates over time.

evaporation The decay of pheromone each iteration; prevents locking onto a suboptimal route early.

NTP Next Transition Probability; the formula an ant uses to pick its next city.

NAS Neural Architecture Search; automated search for the best neural network structure.

NEAT NeuroEvolution of Augmenting Topologies; evolves neural network topology using genetic algorithms.

innovation number A unique ID for each structural mutation; ensures correct alignment during

	crossover.
speciation	Grouping similar architectures so they compete within their group, preserving diversity.
DeepNEAT	NEAT extended so each graph node is a full layer, not a single neuron.
CoDeepNEAT	Co-evolving Deep NEAT; splits architecture into blueprints (structure) and modules (reusable sub-networks).
blueprint	High-level graph in CoDeepNEAT; each node is a module species placeholder.
module	A small reusable sub-network in CoDeepNEAT; defines layer types and hyperparameters.
phenotype	The actual trainable network assembled from a blueprint and a set of modules.
fitness	A score measuring how well a network performs; drives selection.
shared fitness	Distributes a network's fitness score back to both the blueprint and all participating modules.
BLEU score	A metric for text generation quality (0 to 1); higher is better.
ReLU	Rectified Linear Unit; $f(x) = \max(0, x)$.
softmax	Turns a vector of raw scores into probabilities summing to 1.

Symbols at a glance.

Symbol	Meaning
N	Number of ants (must be > 1)
τ_{ij}	Pheromone on edge (i, j); higher = more attractive
τ_0	Initial pheromone level, usually 0
η_{ij}	Heuristic on edge (i, j); $= 1/d_{ij}$ (inverse distance)
α	Weight given to pheromone in NTP
β	Weight given to heuristic (distance) in NTP
ρ	Evaporation coefficient; $0 < \rho < 1$
Q	Pheromone deposit constant
f_k	Tour cost of ant k; shorter tours get higher reward
$\Delta\tau_{ij}^k$	Pheromone deposited by ant k on edge (i, j); $= Q/f_k$ if used
NTP_{ij}	Probability of ant moving from city i to city j
$P(y_i)$	Softmax probability of class y_i
z_i	Logit (raw score) for class i before softmax